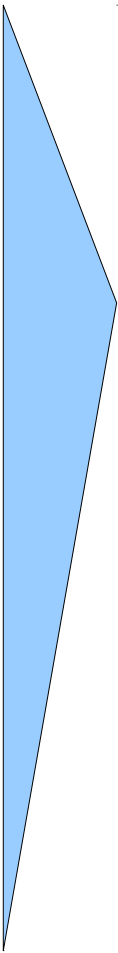
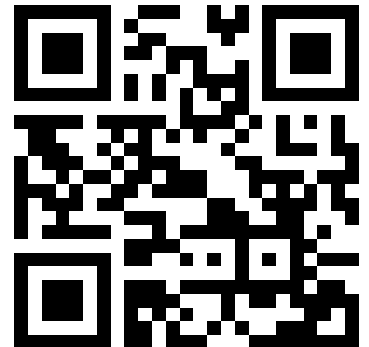
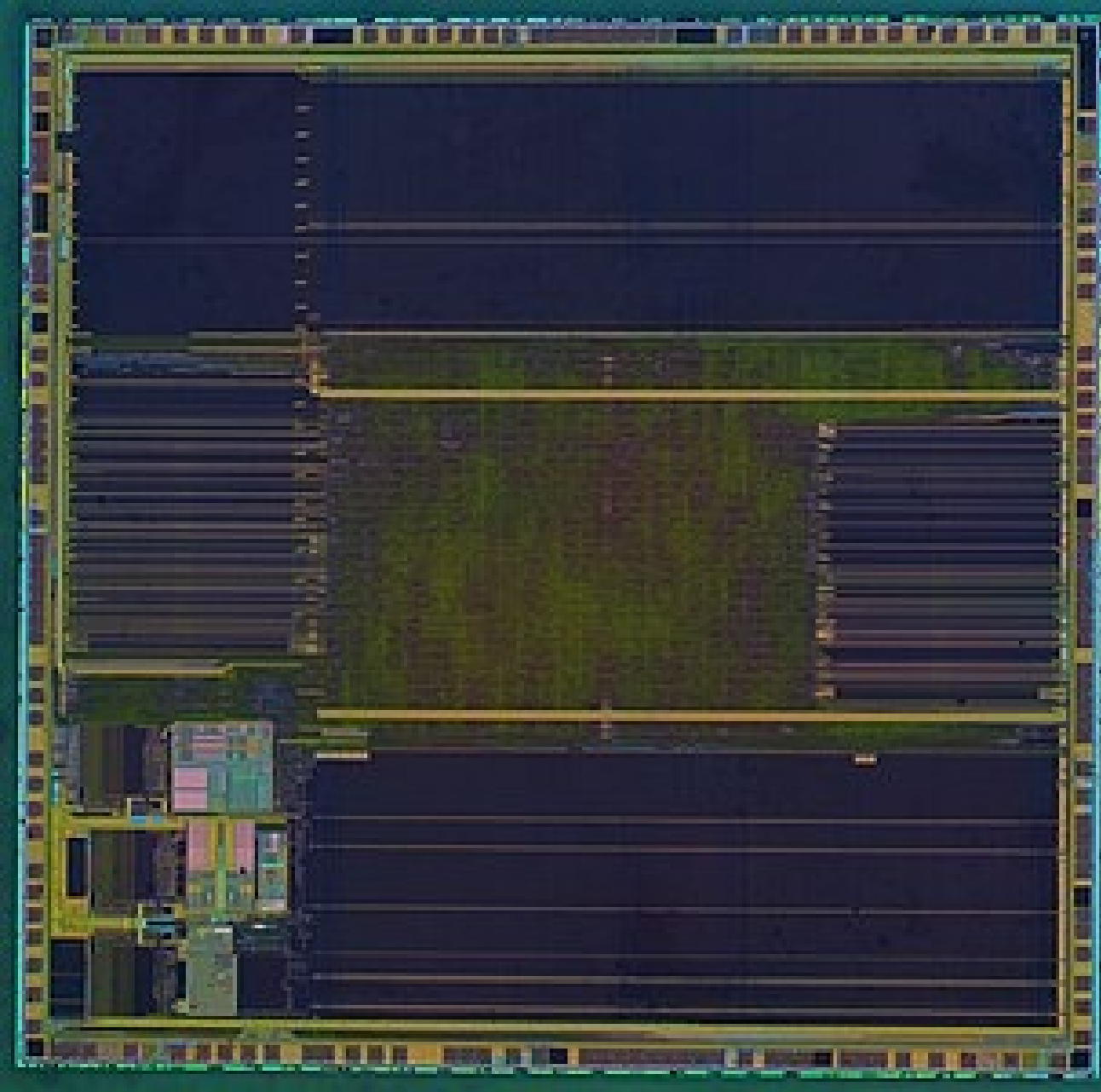


Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>



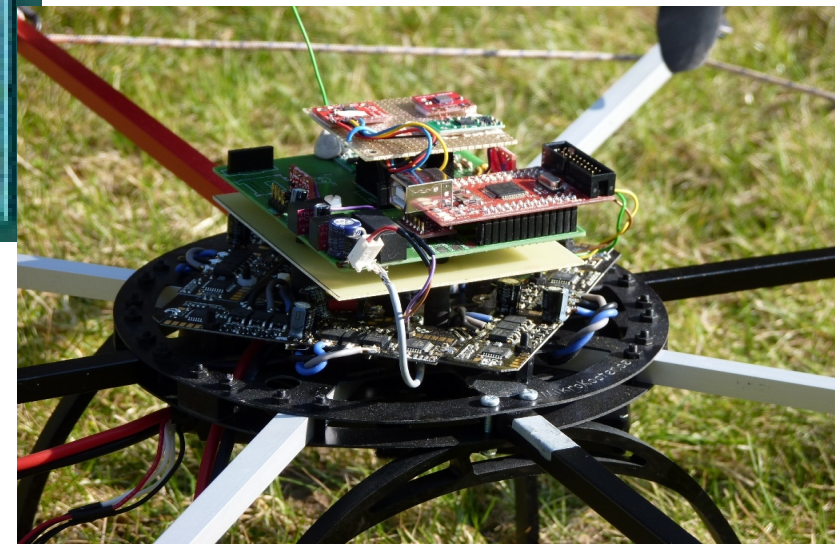


Wikipedia.org STM32F103

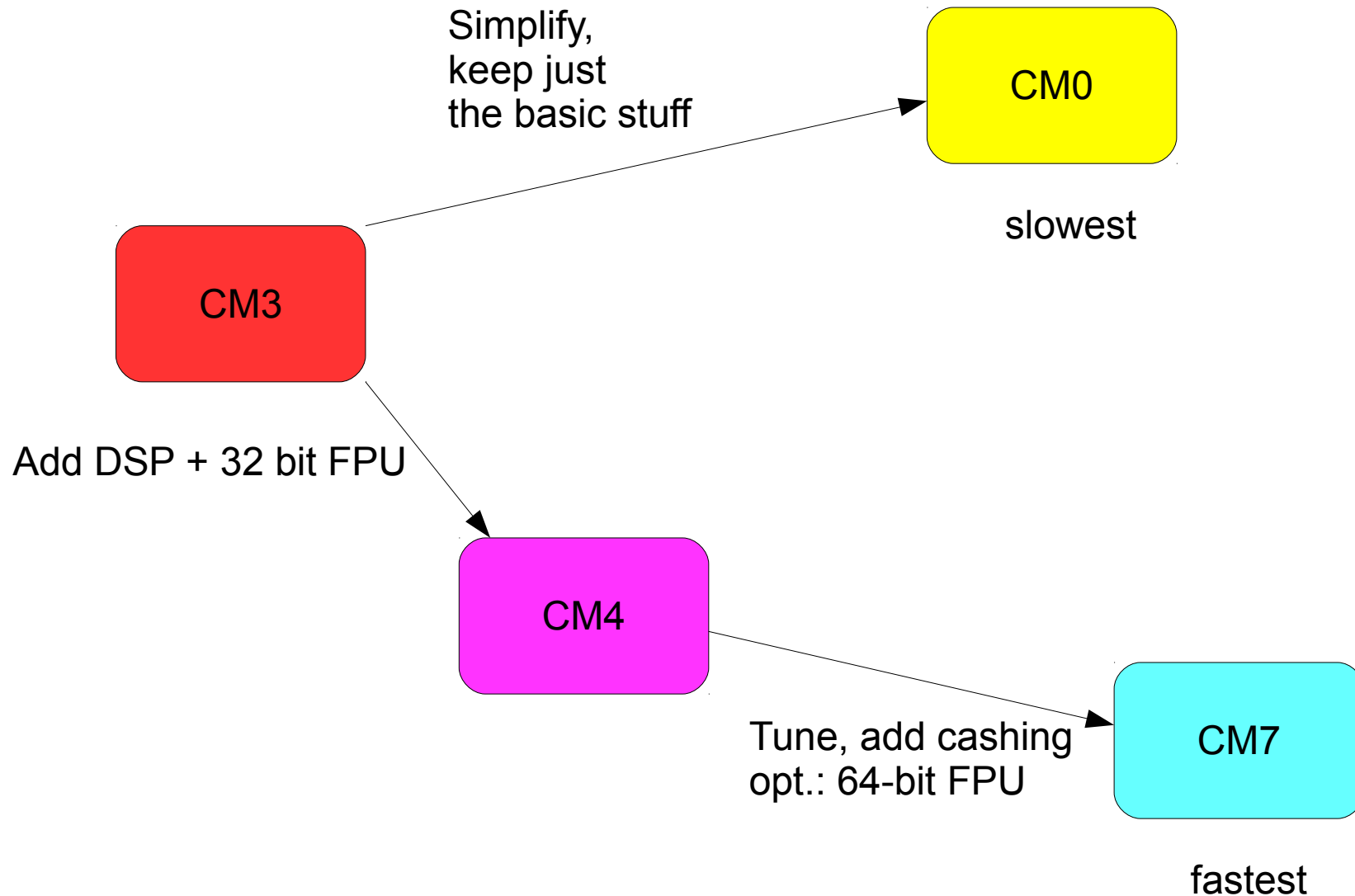


h_da EIT: Blue CAN Node

Colibri FC MK1



ARM Cortex M Family



ARM Cortex M3

Harvard Architecture (= Separate Code and Data buses)

Adds

- various 32 bit Instructions

- Some new addressing modes

- Special data processing commands

- IF / THEN commands

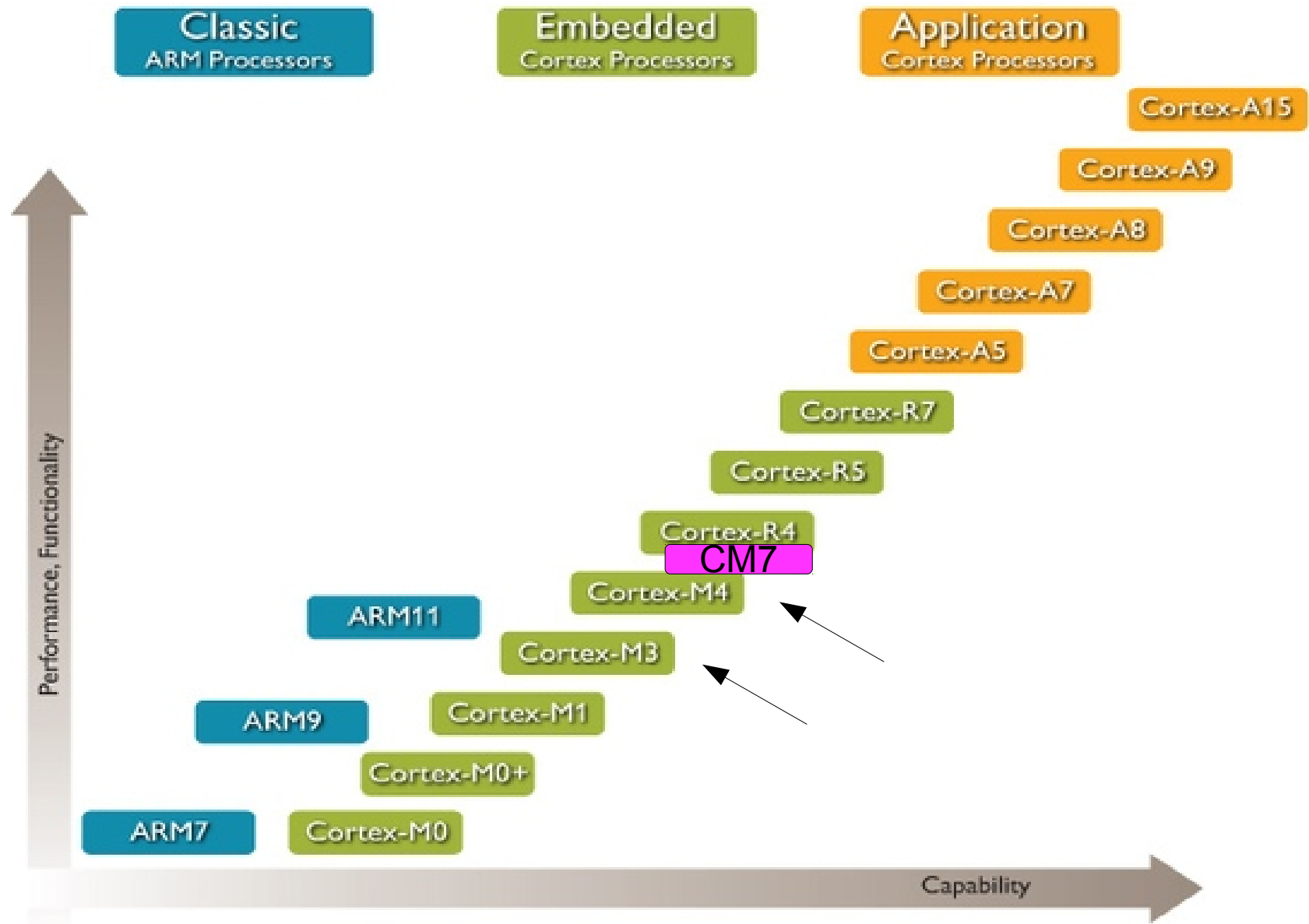
- saturating arithmetics

Unlike the CM0 the CM3 **can** access odd memory locations
Unaligned access is possible, but may be slow.

Please, have a closer look at:

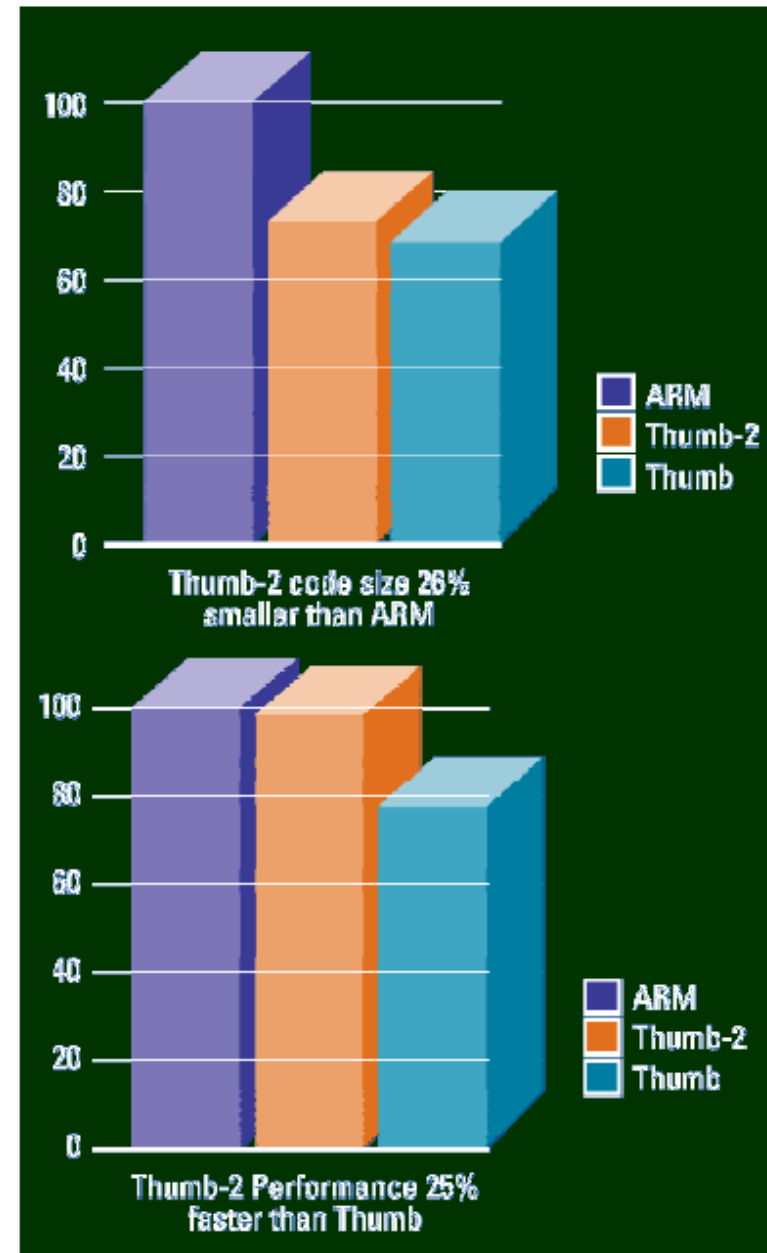
http://skript.eit.h-da.de/ama/Documentation/Cortex_M3_Devices_Generic_User_Guide.pdf

Cortex Controller Family



The Thumb-2 instruction set

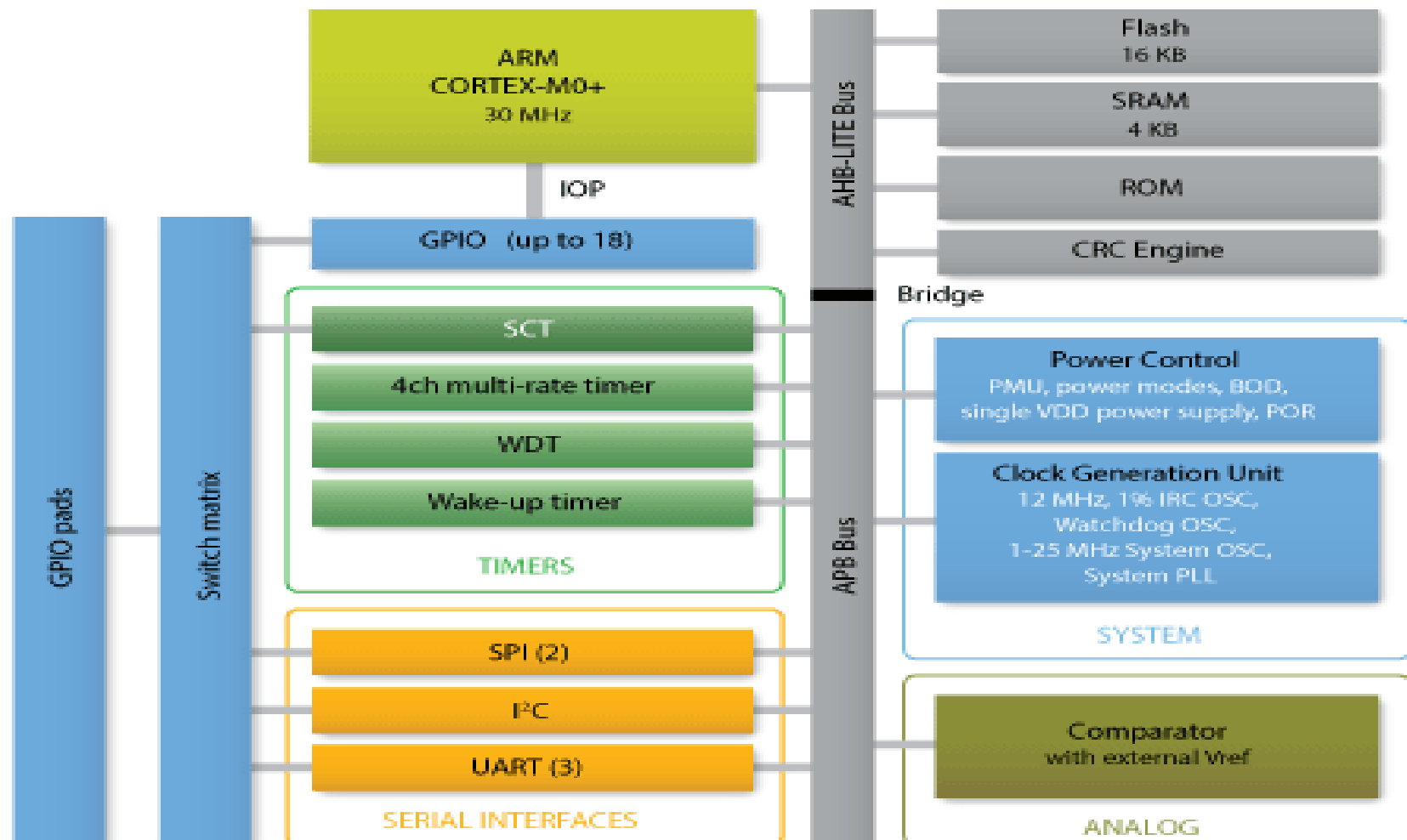
- Variable-length instructions
 - ARM instructions are a fixed length of 32 bits
 - Thumb instructions are a fixed length of 16 bits
 - Thumb-2 instructions can be either 16-bit or 32-bit
- Thumb-2 gives approximately 26% improvement in code density over ARM
- Thumb-2 gives approximately 25% improvement in performance over Thumb



CM0+

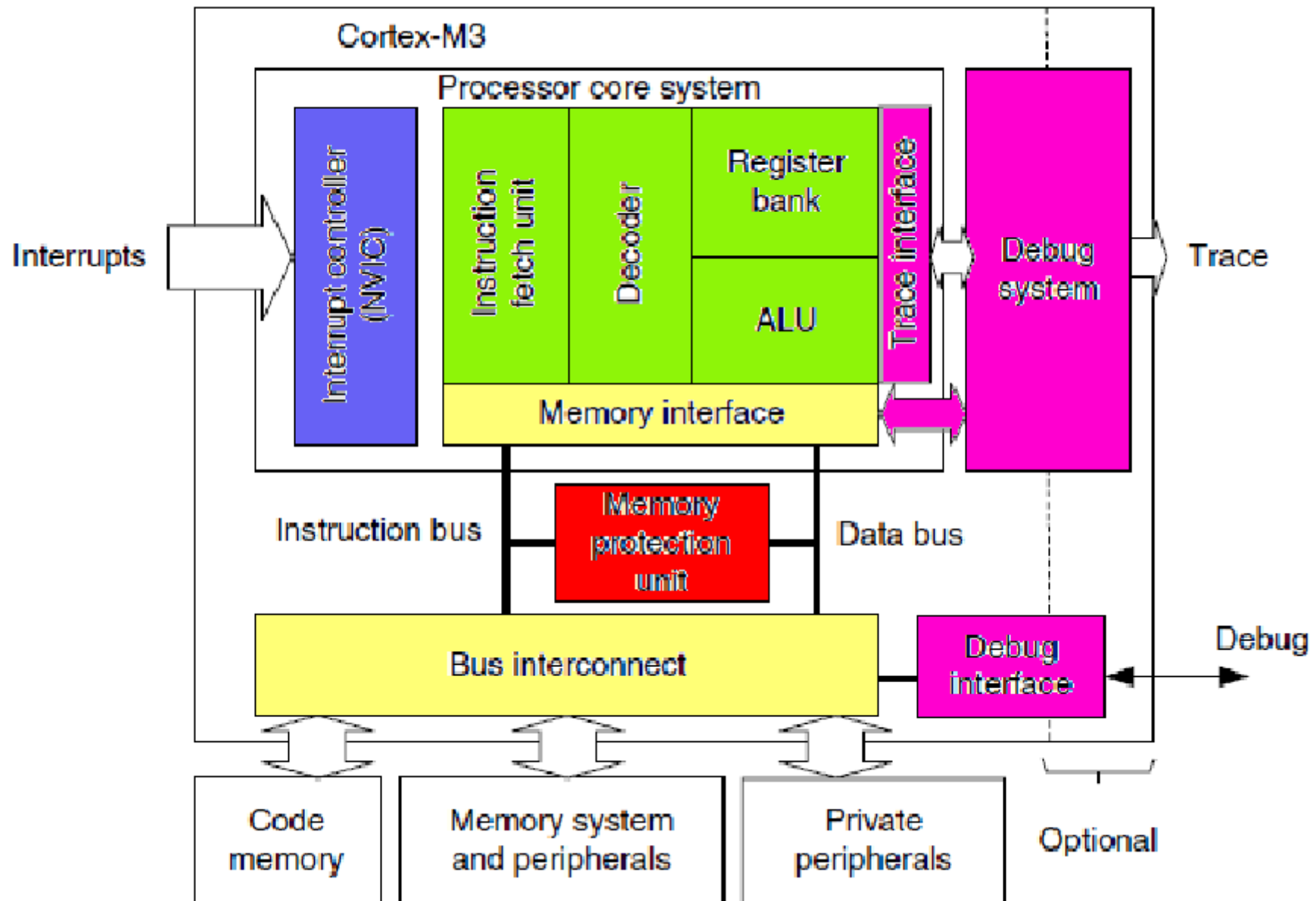
Von Neumann MPU

LPC800: Low Power Cortex-M0+

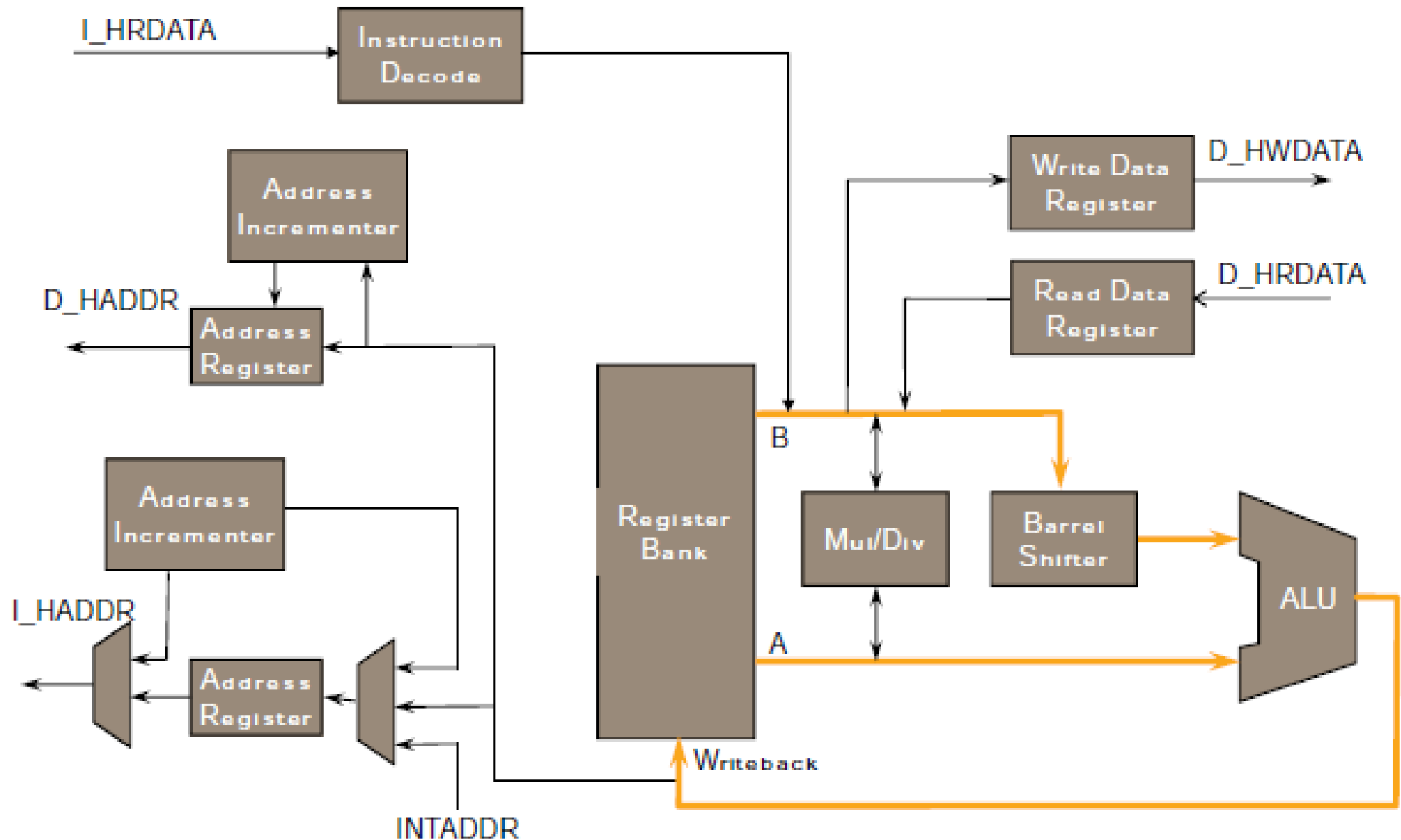


CORTEX M3 Core

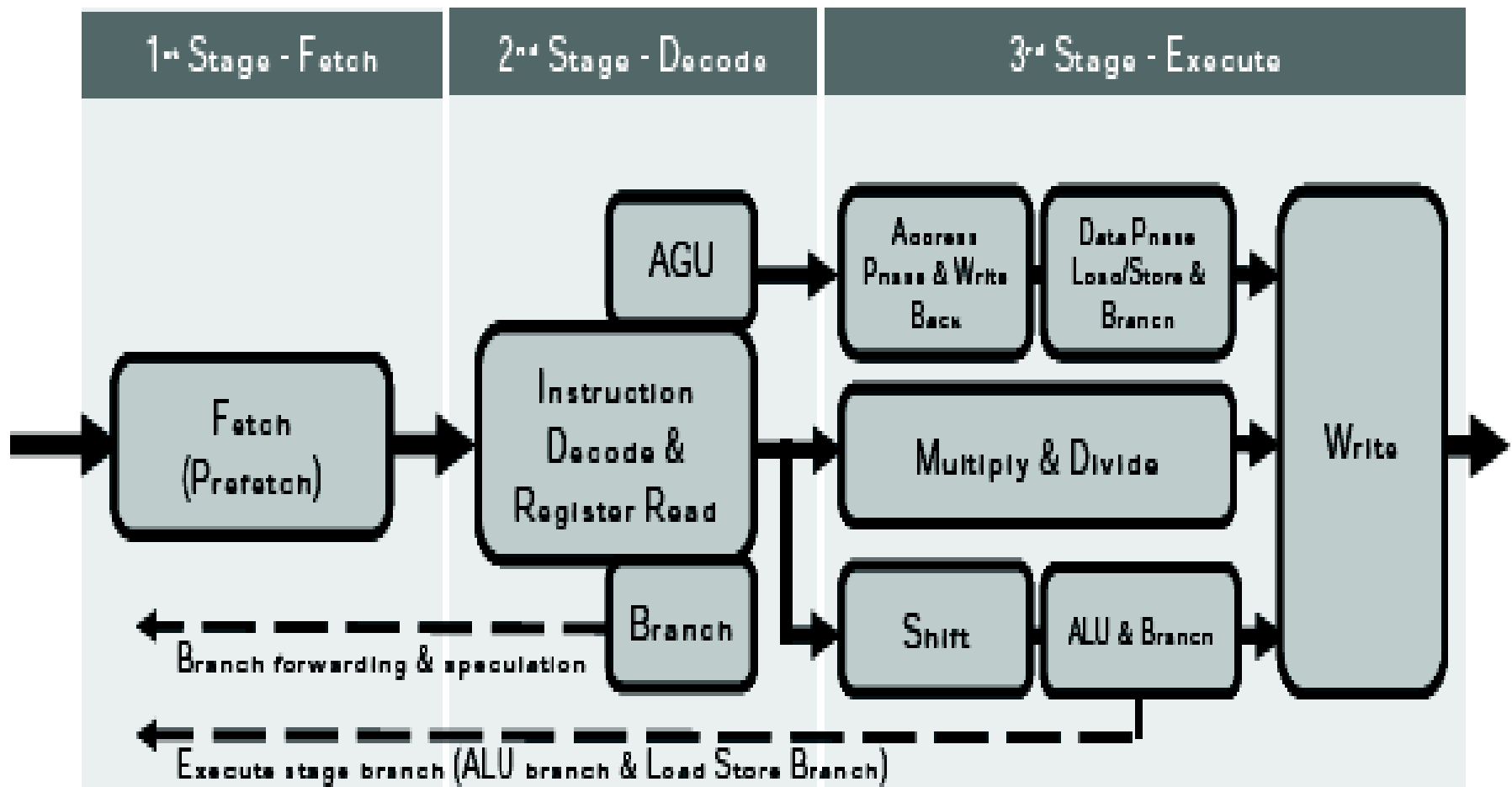
Harvard MPU Architecture



CM3 Data Path

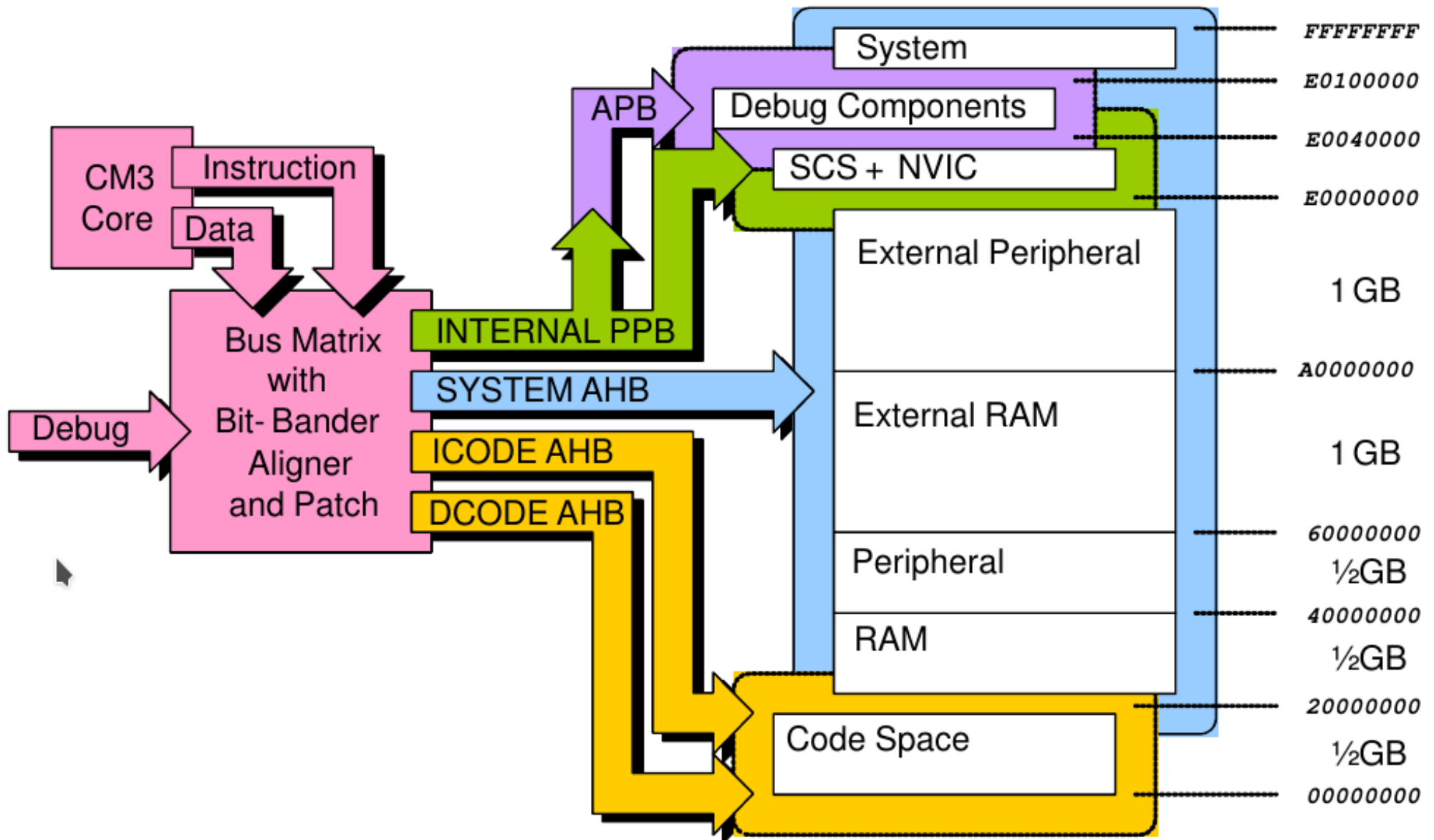


3 Stage Pipeline

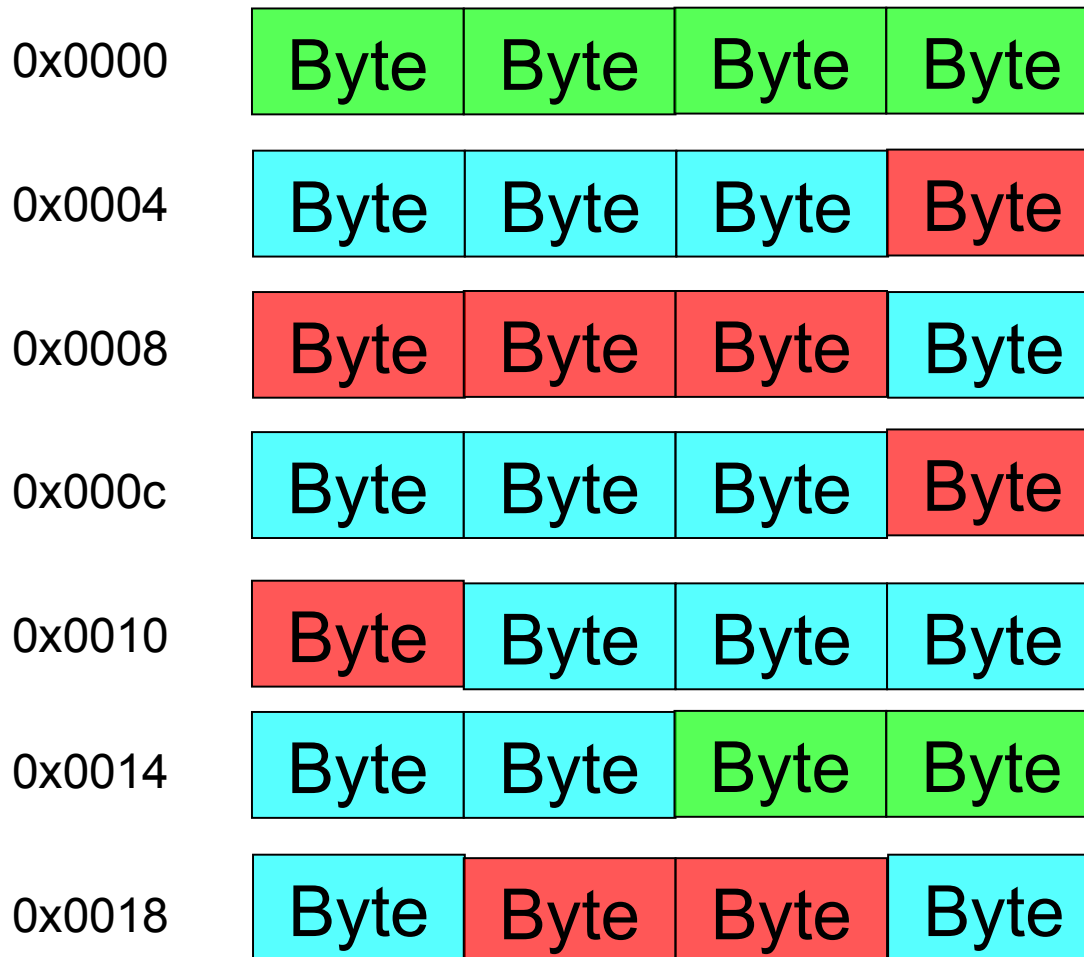


Remark: The CM0+ has a 2 stage pipeline only

CM3 Memory Architecture



Aligned and Unaligned Memory Access



Memory addresses address **bytes**

Memory is organized in 32 bit words

Unaligned access is inefficient
(impossible on CM0's)

For halfword-access
the LSB must be zero

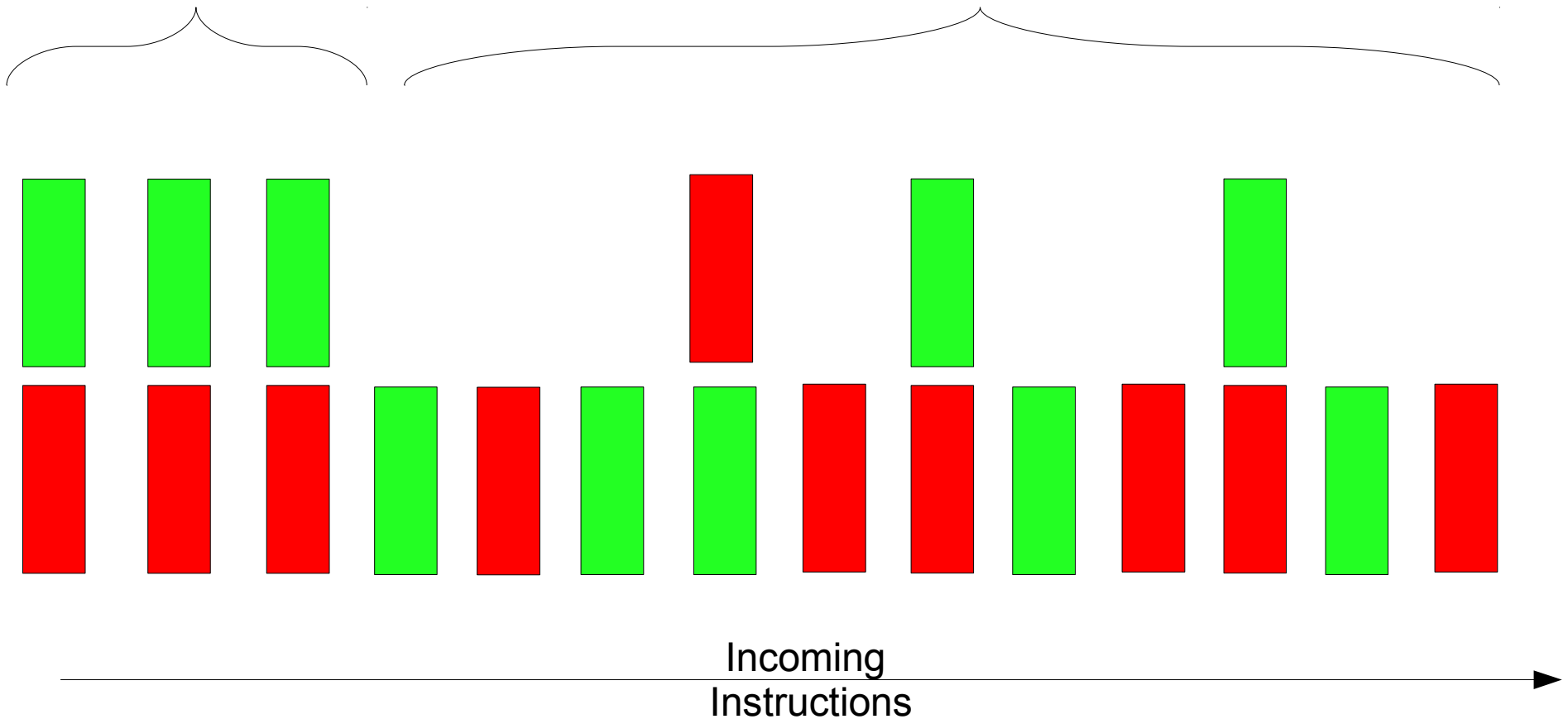
For word-access two LSB bits
must be zero

“even addresses”

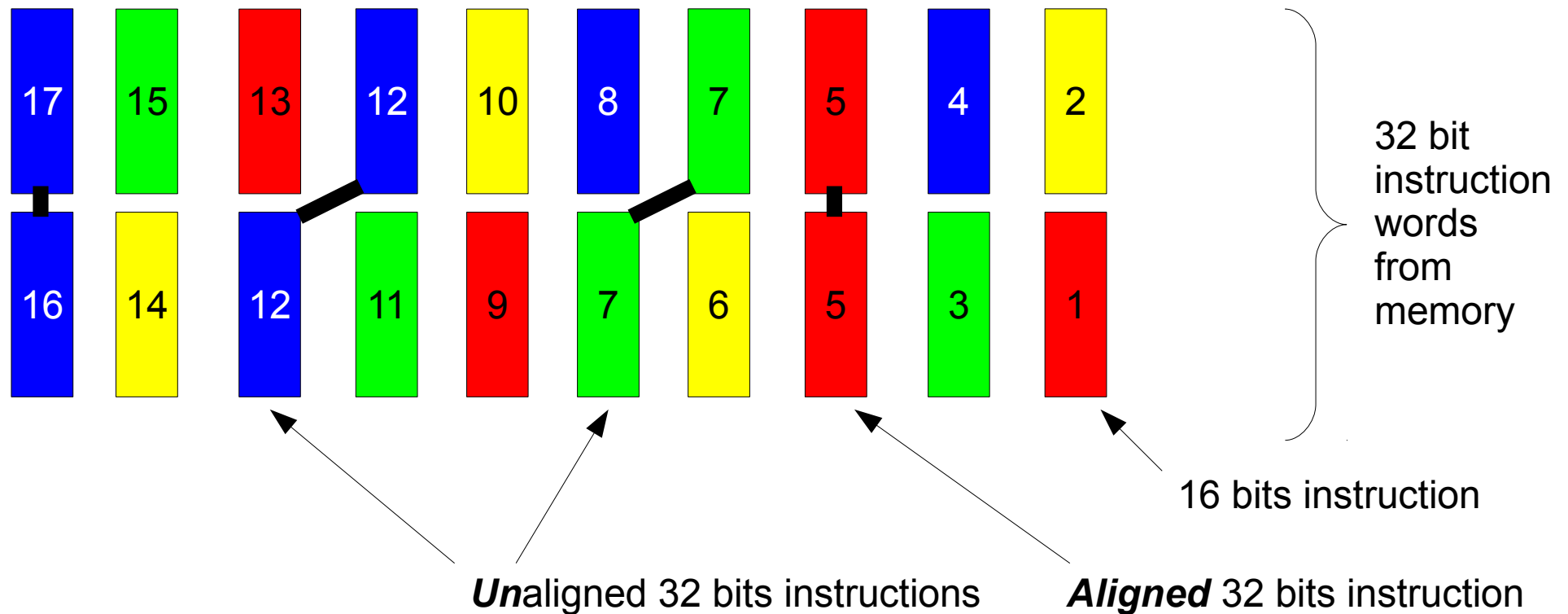
Instruction Buffering

Instruction Fetch
Always 32 bits

Instruction execution
Mixed 16 and 32 bits



Instruction Fetching



The Program counter represents the point to fetch new instructions from
I.E. the PC points **4 bytes ahead** the current instruction

LSB of PC is **always 1** ("Thumb instruction")

Instructions addresses are always even ("no 8-bit instructions out there")

32 bits instructions may be unaligned
(Split into two halves of two different 32 bit instruction words)

Optimal Pipelining

Cycle		1	2	3	4	5	6	7	8	9
Operation										
ADD	F	D	E							
SUB		F	D	E						
ORR			F	D	E					
AND				F	D	E				
ORR					F	D	E			
EOR						F	D	E		

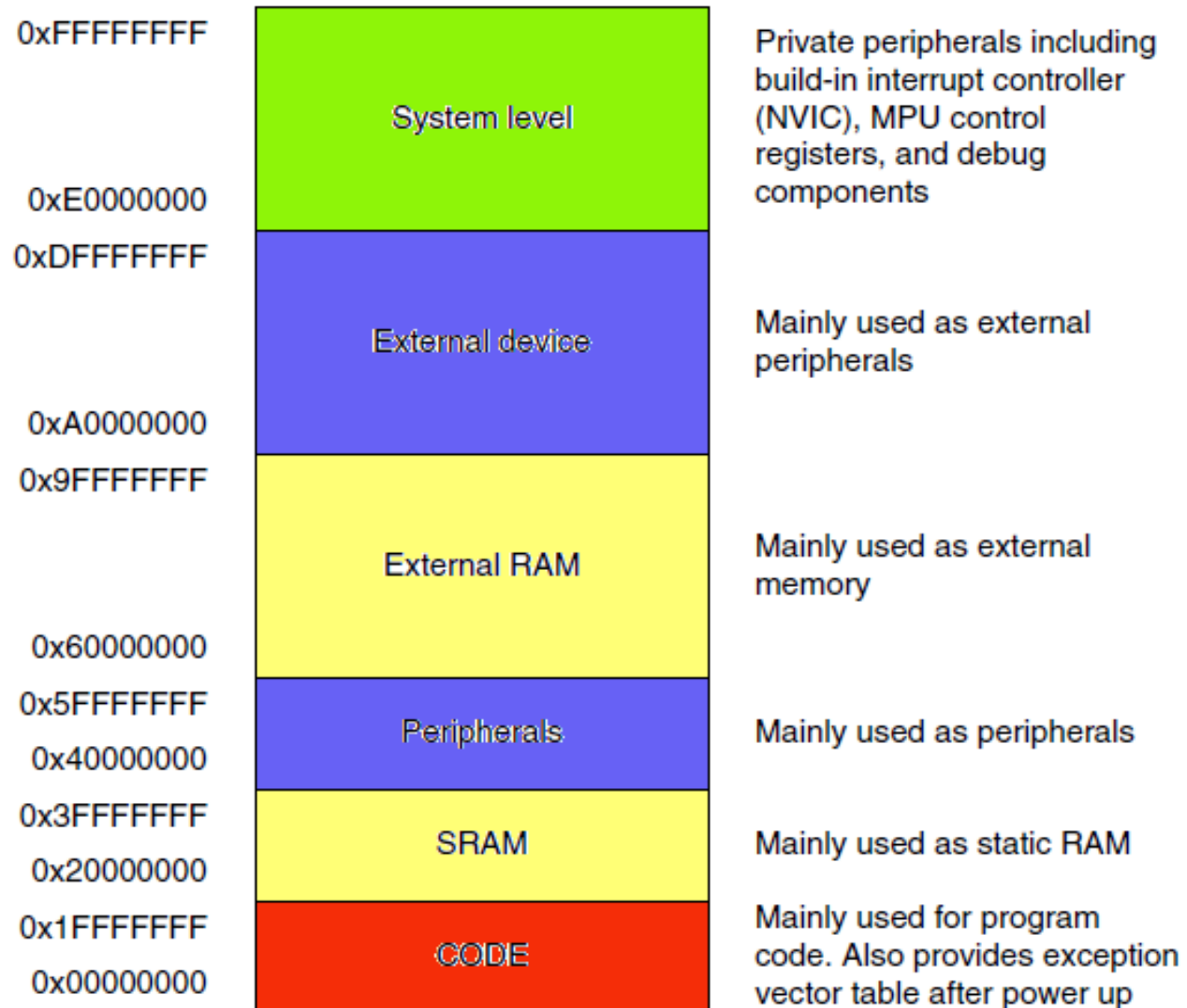
F - Fetch D - Decode E - Execute

- All operations here are on registers (single cycle execution)
- In this example it takes 6 clock cycles to execute 6 instructions
- Clock cycles per Instruction (CPI) = 1

Register Set

Name		Functions (and banked registers)	
R0		General-purpose register	Low registers
R1		General-purpose register	
R2		General-purpose register	
R3		General-purpose register	
R4		General-purpose register	
R5		General-purpose register	
R6		General-purpose register	
R7		General-purpose register	High registers
R8		General-purpose register	
R9		General-purpose register	
R10		General-purpose register	
R11		General-purpose register	
R12		General-purpose register	
R13 (MSP)	R13 (PSP)	Main Stack Pointer (MSP), Process Stack Pointer (PSP)	
R14		Link Register (LR)	
R15		Program Counter (PC)	

CM3 / CM4 Memory Map



Program Status Register (Flags)

	31	30	29	28	27	26:25	24	23:20	19:16	15:10	9	8	7	6	5	4:0
APSR	N	Z	C	V	Q											
IPSR												Exception number				
EPSR						IC/IT		T				IC/IT				

$XPSR = APSR \mid IPSR \mid EPSR$

Reading / Writing flags:

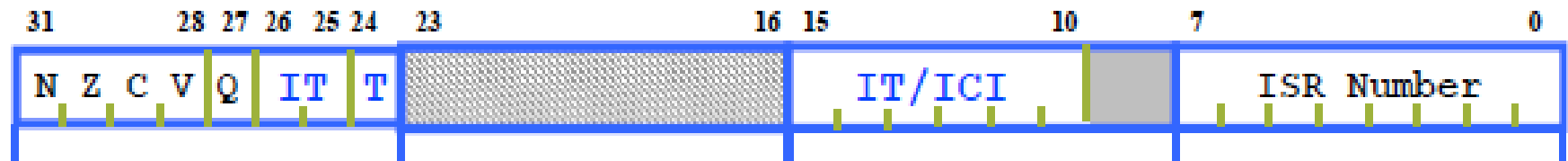
MRS r0, APSR ; Move to Register from Special register
MRS r0, IPSR
MRS r0, EPSR

MSR APSR,r0 ; Move to Special from Register

Interrupt-continuable
Instruction.
If / Then

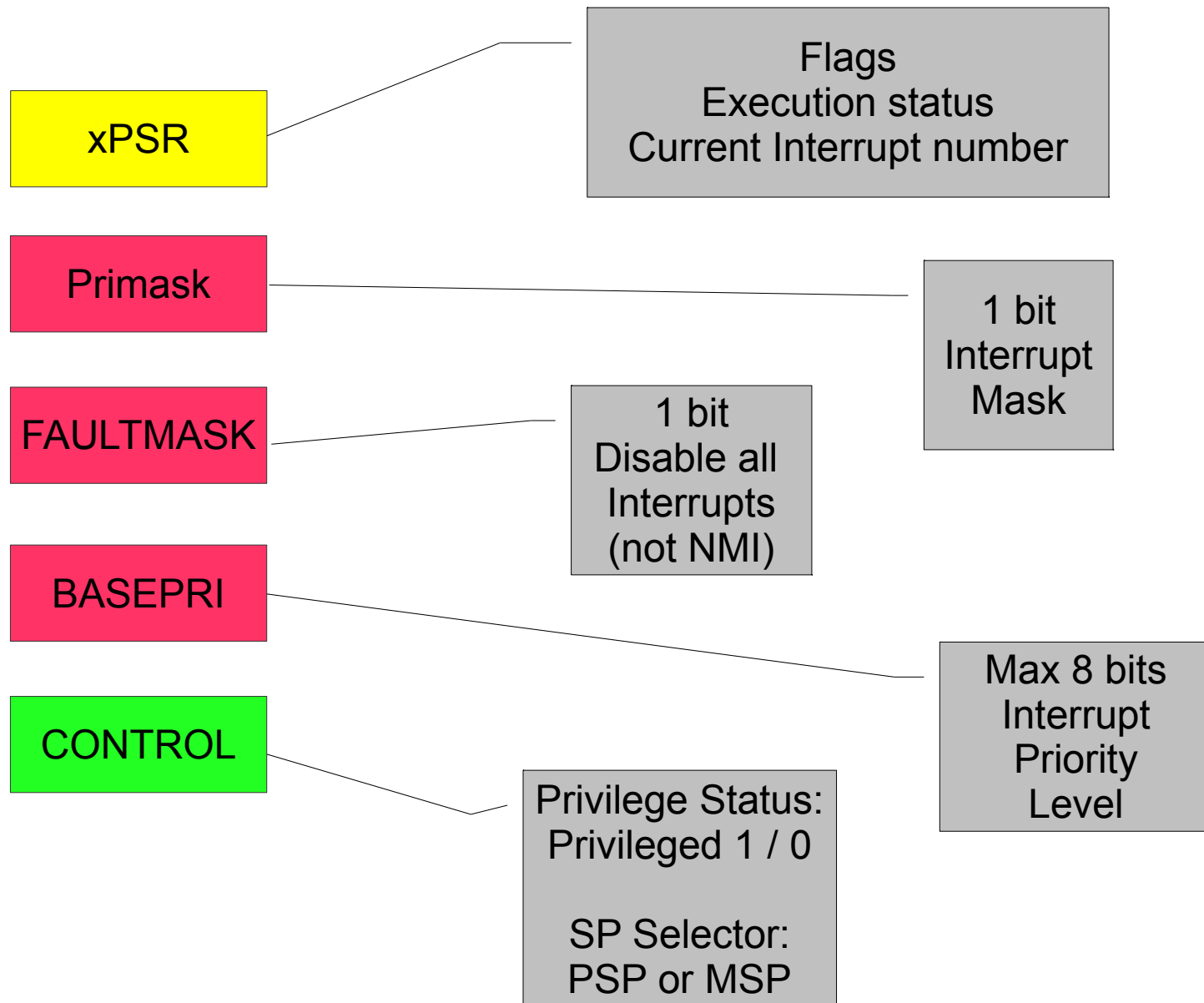
● Q = "Sticky saturation flag"

Cortex-M3 Program Status Register



- One Status Register consisting of
 - APSR - Application Program Status Register – ALU flags
 - IPSR - Interrupt Program Status Register – Interrupt/Exception No.
 - EPSR - Execution Program Status Register
 - IT field – If/Then block information
 - ICI field – Interruptible-Continuable Instruction information
- xPSR
 - Composite of the 3 PSRs
 - Stored on the stack on exception entry

Special “Registers”



Cortex-M4 FPU

VABS	VADD	VCMP	VCMPE	VCVT	VCVTR	VDIV	VLDM
VLDR	VMLA	VMLS	VMOV	VMRS	VMSR	VMUL	VNEG
VNMLA	VMMLS	VNMUL	VPOP	VPUSH	VSQRT	VSTM	VSTR
VSUB	VFMA	VFMS	VFNMA	VFNMS			

PKH	QADD	QADD16	QADD8	QASX	QDADD	QDSUB	QSAX
QSUB	QSUB16	QSUB8	SADD16	SADD8	SASX	SEL	SHADD16
SHADD8	SHASX	SHSAX	SHSUB16	SHSUB8	SMLABB	SMLABT	SMLATB
SMLATT	SMLAD	SMLALBB	SMLALBT	SMLALTB	SMLALTT	SMLALD	SMLAWB
SMLAWT	SMLSD	SMLSLD	SMMLA	SMMLS	SMMUL	SMUAD	SMULBB

ADC	ADD	ADR	AND	ASR	B	SMULBT	SMULTT
CLZ	BFC	BFI	BIC	CDP	CLREX	SMULTB	SMULWT
CBNZ	CBZ	CMN	CMP	DBG	EOR	SMULWB	SMUSD
LDMIA	LDMDB	LDR	LDRB	LDRBT	LDRD	SSAT16	SSAX
LDREX	LDREXB	LDREXH	LDRH	LDRHT	LDRSB	SSUB16	SSUB8
LDRSBT	LDRSHT	LDRSH	LDRT	MCR	LSL	SXTAB	SXTAB16
LSR	MCRR	MLS	MLA	MOV	MOVT	SXTAH	SXTB16
MRC	MRRC	MUL	MVN	NOP	ORN	UADD16	UADD8
ORR	PLD	PLDW	PLI	POP	PUSH	UASX	UHADD16
RBIT	REV	REV16	REVSH	ROR	RRX	UHADD8	UHASX
			RSB	SBC	SBFX	UHSAX	UHSUB16
			SDIV	SEV	SMLAL	UHSUB8	UMAAL
			SMULL	SSAT	STC	UQADD16	UQADD8
			STMIA	STMDB	STR	UQASX	UQSAX
			STRB	STRBT	STRD	UQSUB16	UQSUB8
			STREX	STREXB	STREXH	USAD8	USADA8
			STRH	STRHT	STRT	USAT16	USAX
			SUB	SXTB	SXTH	USUB16	USUB8
			TBB	TBH	TEQ	UXTAB	UXTAB16
			TST	UBFX	UDIV	UXTAH	UXTB16
			UMLAL	UMULL	USAT		
			UXTB	UXTH	WFE		
			WFI	YIELD	IT		

BKPT	BLX	ADC	ADD	ADR
BX	CPS	AND	ASR	B
DMB		BL		BIC
DSB	CMN	CMP	EOR	
ISB	LDR	LDRB	LDM	
MRS	LDRH	LDRSB	LDRSH	
MSR	LSL	LSR	MOV	
NOP	REV	MUL	MVN	ORR
REV16	REVSH	POP	PUSH	ROR
SEV	SXTB	RSB	SBC	STM
SXTH	UXTB	STR	STRB	STRH
UXTH	WFE	SUB	SVC	TST
WFI	YIELD			

Cortex-M0/M0+/M1

Cortex-M3

Cortex-M4

ARM Cortex M3

Includes all CM0 Instructions

Adds

- various 32 bit Instructions

- addressing modes

- data processing commands

- IF / THEN / ELSE - instructions

- saturating arithmetics

Unlike the CM0 the CM3 **can** access odd memory locations
Unaligned access is possible, but may be slow.

Subroutine Call and Return

```
...  
; R0 = X, R1 = Y, R2 = Z  
BL    function1
```

Subroutine

function1

```
    PUSH    {R0} ; store R0 to stack & adjust SP  
    PUSH    {R1} ; store R1 to stack & adjust SP  
    PUSH    {R2} ; store R2 to stack & adjust SP  
    ... ; Executing task (R0, R1 and R2  
        ; could be changed)  
    POP     {R2} ; restore R2 and SP re-adjusted  
    POP     {R1} ; restore R1 and SP re-adjusted  
    POP     {R0} ; restore R0 and SP re-adjusted  
    BX      LR   ; Return
```

```
; Back to main program  
; R0 = X, R1 = Y, R2 = Z  
... ; next instructions
```

- Descending Stack
- The stack-pointer points to the last used datum on the stack
- The Stack is always word-aligned, i.e. the last two bits of SP are always zero

PUSH / POP

Multi - Registers

Syntax:

PUSH {cond} reglist
POP {cond} reglist

Examples:

PUSH {R0,R4-R7} ; Push R0,R4,R5,R6,R7 onto the stack

PUSH {R2,LR} ; Push R2 and the link-register onto the stack

POP {R0,R6,PC} ; Pop r0,r6 and PC from the stack, then branch to the new PC.

Restrictions

In these instructions:

- reglist must not contain SP

- for the PUSH instruction, reglist must not contain PC

- for the POP instruction, reglist must not contain PC if it contains LR.

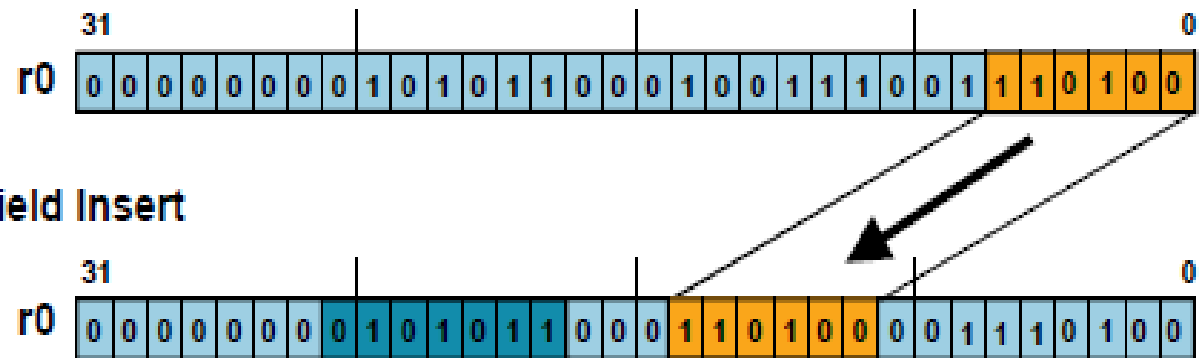
When PC is in reglist in a POP instruction:

- bit[0] of the value loaded for PC must be 1 for correct execution

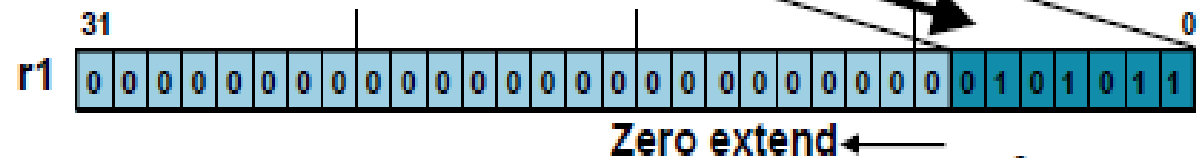
- if the instruction is conditional, it must be the last instruction in the IT block.

Bit Manipulation Instructions

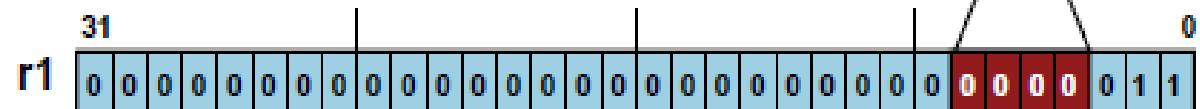
BFI r0, r0, #9, #6 ; Bit Field Insert



UBFX r1, r0, #18, #7 ; Bit Field Extract



BFC r1, #3, #4 ; Bit Field Clear



RBIT r2, r1 ; Reverse Bit Order



Bit Clear / Insert

Syntax

`BFC{cond} Rd, #lsb, #width`

`BFI{cond} Rd, Rn, #lsb, #width`

where:

cond

Is an optional condition code, see [Conditional execution](#).

Rd

Specifies the destination register.

Rn

Specifies the source register.

lsb

Specifies the position of the least significant bit of the bitfield. *lsb* must be in the range 0 to 31.

width

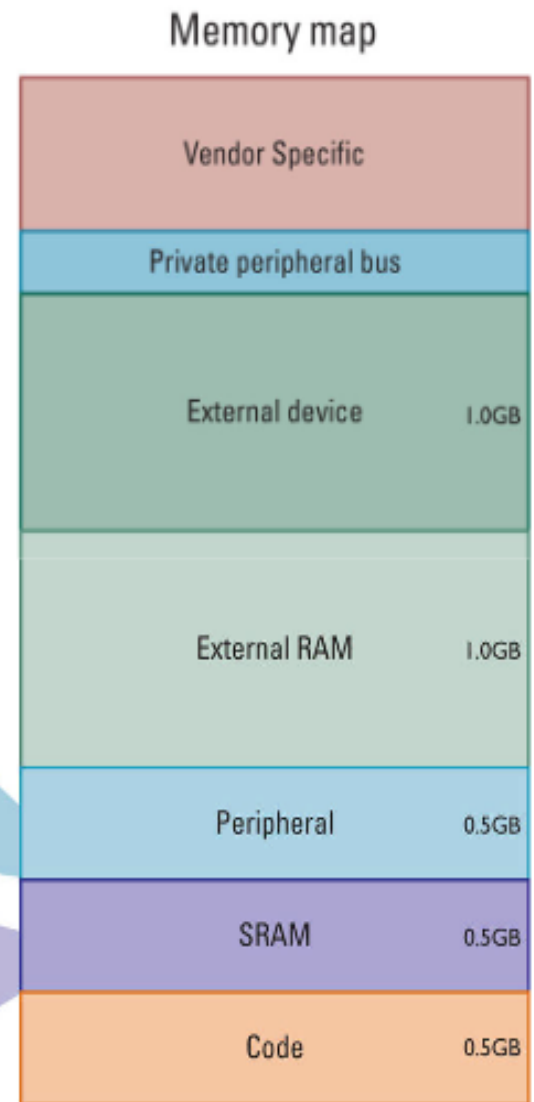
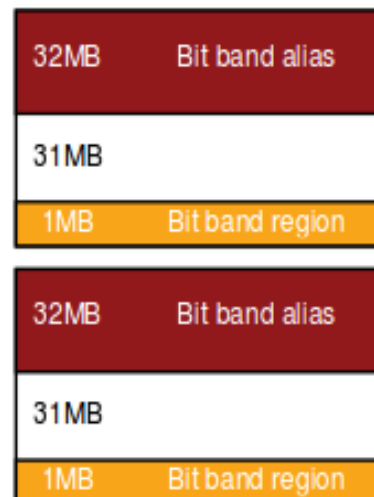
Specifies the width of the bitfield and must be in the range 1 to 32-*lsb*.

Examples

```
BFC    R4, #8, #12    ; Clear bit 8 to bit 19 (12 bits) of R4 to 0
BFI    R9, R2, #8, #12 ; Replace bit 8 to bit 19 (12 bits) of R9 with
                        ; bit 0 to bit 11 from R2.
```

Cortex-M3 Bit Banding

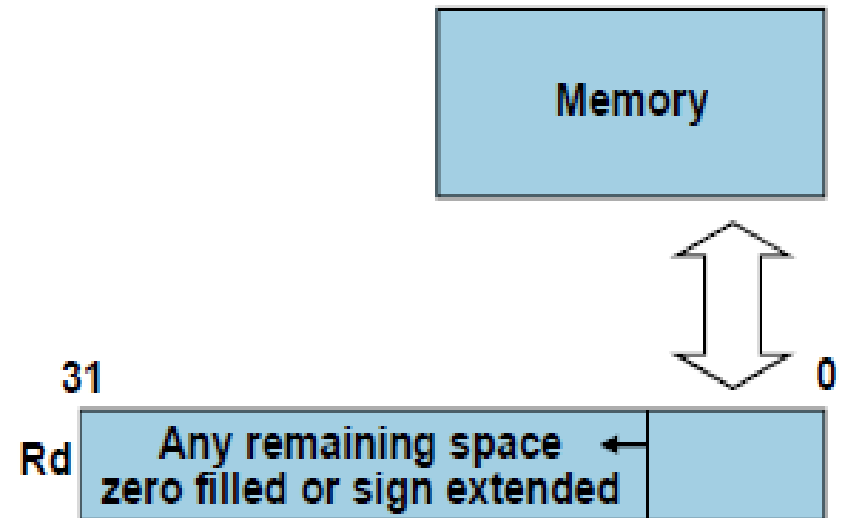
- Writes to a word address in the bit band alias affect a single bit in the bit band region
- The write is translated to an atomic read-modify-write by the Cortex-M3 bus matrix
- Bit 0 of the stored register is written to the appropriate bit



Single / Double Register Data Transfer

- Use to move data between one or two registers and memory

LDRD	STRD	Doubleword
LDR	STR	Word
LDRB	STRB	Byte
LDRH	STRH	Halfword
LDRSB		Signed byte load
LDRSH		Signed halfword load



- Syntax

- **LDR**{<size>}{<cond>} Rd, <address>
- **STR**{<size>}{<cond>} Rd, <address>

LDR and STR

Syntax

<code>op{type}{cond} Rt, [Rn {, #offset}]</code>	<code>; immediate offset</code>
<code>op{type}{cond} Rt, [Rn, #offset]!</code>	<code>; pre-indexed</code>
<code>op{type}{cond} Rt, [Rn], #offset</code>	<code>; post-indexed</code>
<code>opD{cond} Rt, Rt2, [Rn {, #offset}]</code>	<code>; immediate offset, two words</code>
<code>opD{cond} Rt, Rt2, [Rn, #offset]!</code>	<code>; pre-indexed, two words</code>
<code>opD{cond} Rt, Rt2, [Rn], #offset</code>	<code>; post-indexed, two words</code>

where:

op

Is one of:

LDR

Load Register.

STR

Store Register.

type

Is one of:

B

unsigned byte, zero extend to 32 bits on loads.

SB

signed byte, sign extend to 32 bits (LDR only).

H

unsigned halfword, zero extend to 32 bits on loads.

SH

signed halfword, sign extend to 32 bits (LDR only).

LDR and STR

Examples

LDR	R8, [R10]	; Loads R8 from the address in R10.
LDRNE	R2, [R5, #960]!	; Loads (conditionally) R2 from a word ; 960 bytes above the address in R5, and ; increments R5 by 960
STR	R2, [R9, #const-struct]	; const-struct is an expression evaluating ; to a constant in the range 0-4095.
STRH	R3, [R4], #4	; Store R3 as halfword data into address in ; R4, then increment R4 by 4
LDRD	R8, R9, [R3, #0x20]	; Load R8 from a word 8 bytes above the ; address in R3, and load R9 from a word 9 ; bytes above the address in R3
STRD	R0, R1, [R8], #-16	; Store R0 to address in R8, and store R1 to ; a word 4 bytes above the address in R8, ; and then decrement R8 by 16.

LDR and STR

PC - Relative

Syntax

`LDR{type}{cond} Rt, label`

`LDRD{cond} Rt, Rt2, label` ; Load two words

where:

type

Is one of:

B

unsigned byte, zero extend to 32 bits.

SB

signed byte, sign extend to 32 bits.

H

unsigned halfword, zero extend to 32 bits.

SH

signed halfword, sign extend to 32 bits.

-

omit, for word.

Examples

```
LDR    R0, LookUpTable    ; Load R0 with a word of data from an address
                          ; labelled as LookUpTable
LDRSB  R7, localdata      ; Load a byte value from an address labelled
                          ; as localdata, sign extend it to a word
                          ; value, and put it in R7.
```

Single register data transfer

LDR	STR	Word
LDRB	STRB	Byte
LDRH	STRH	Halfword
LDRSB		Signed byte load
LDRSH		Signed halfword load

- Memory system must support all access sizes
- Syntax:
 - LDR{<cond>}{<size>} Rd, <address>
 - STR{<cond>}{<size>} Rd, <address>

e.g. LDREQB

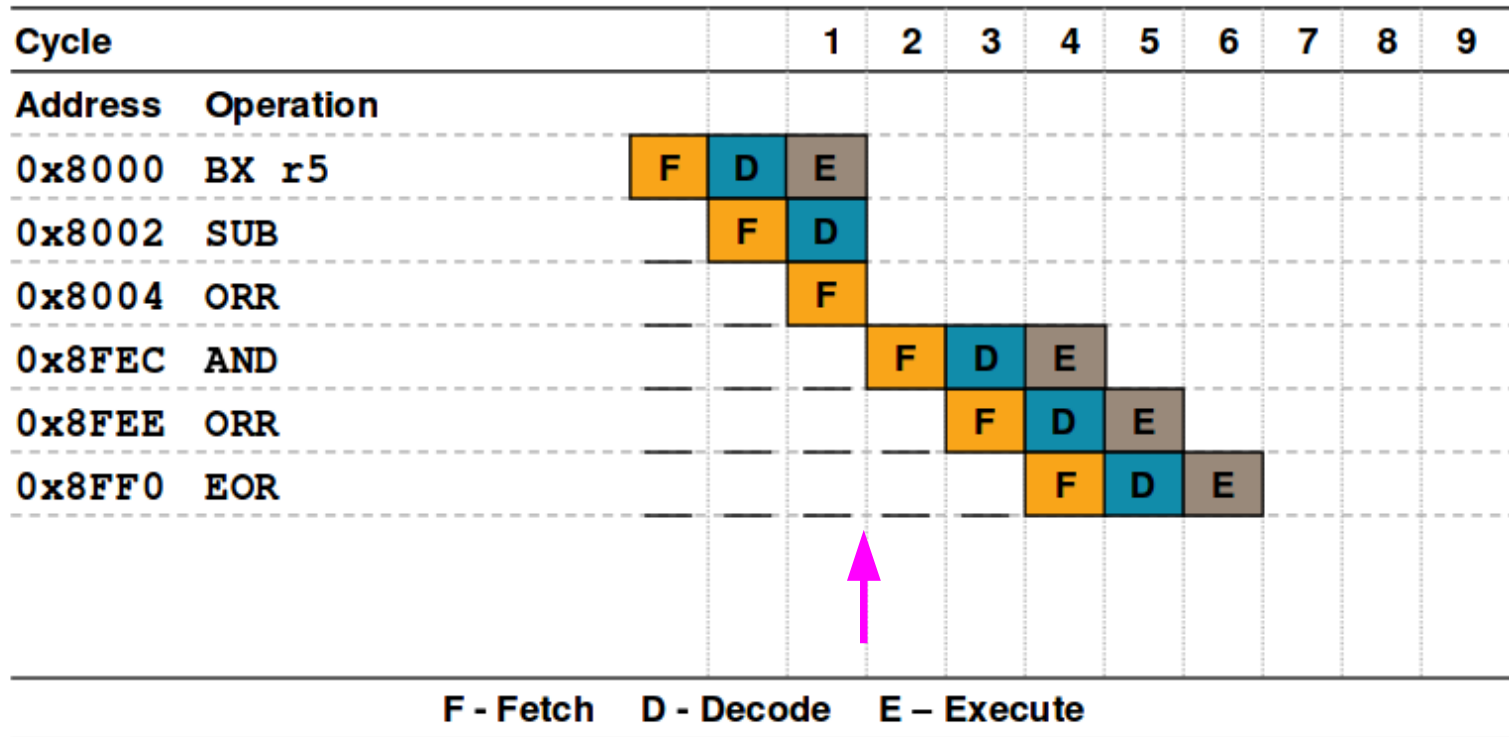
LDR Pipeline Example

Cycle		1	2	3	4	5	6	7	8	9
Operation										
ADD	F	D	E							
SUB		F	D	E						
LDR			F	D	Ea	Ed				
AND				F	D	S	E			
ORR					F	S	D	E		
EOR						F	D	E		

F - Fetch **D** - Decode **E** - Execute **S** - Stall
Ea - LDR address phase **Ed** - LDR data phase

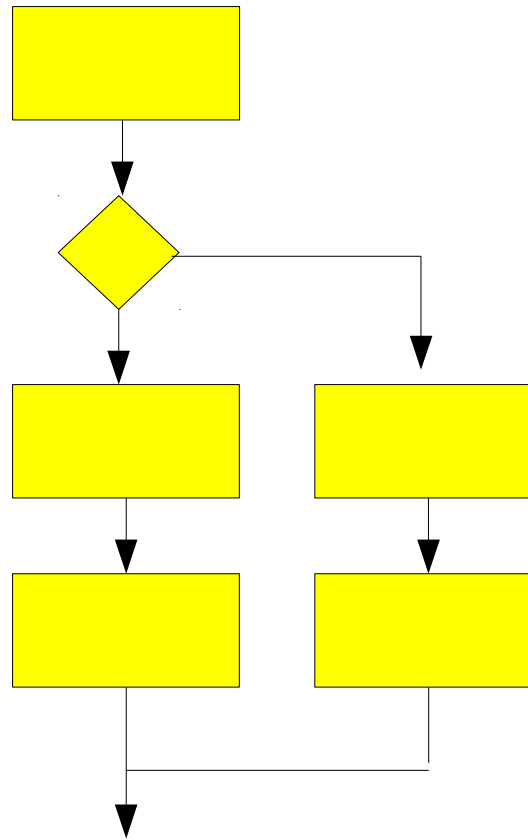
- In this example it takes 7 clock cycles to execute 6 instructions
- Clock cycles per Instruction (CPI) = 1.2
- The read cycle must complete on the bus before the LDR instruction can complete since there is only one writeback port in the register file

Branch Pipeline Example



- It takes 3 cycles to complete the branch
- Worst case scenario – indirect branch taken

"Branch Penalty"



If the processor executes a branch
pre-fetched instructions become useless.
Obviously this slows down the program execution.

Quite frequently this can be avoided by using
the ITE instruction group for short if / then sequences

If Then - Instructions

Syntax

`IT{x{y{z}}} cond`

where:

x

specifies the condition switch for the second instruction in the IT block.

y

specifies the condition switch for the third instruction in the IT block.

z

specifies the condition switch for the fourth instruction in the IT block.

cond

specifies the condition for the first instruction in the IT block.

The condition switch for the second, third and fourth instruction in the IT block can be either:

T

Then. Applies the condition *cond* to the instruction.

E

Else. Applies the inverse condition of *cond* to the instruction.

Example

```
ITTE    NE                ; Next 3 instructions are conditional
ANDNE   R0, R0, R1        ; ANDNE does not update condition flags
ADDsNE  R2, R2, #1        ; ADDsNE updates condition flags
MOVEQ   R2, R3            ; Conditional move

CMP      R0, #9           ; Convert R0 hex value (0 to 15) into ASCII
                          ; ('0'-'9', 'A'-'F')
ITE      GT              ; Next 2 instructions are conditional
ADDGT   R1, R0, #55       ; Convert 0xA -> 'A'
ADDLE   R1, R0, #48       ; Convert 0x0 -> '0'

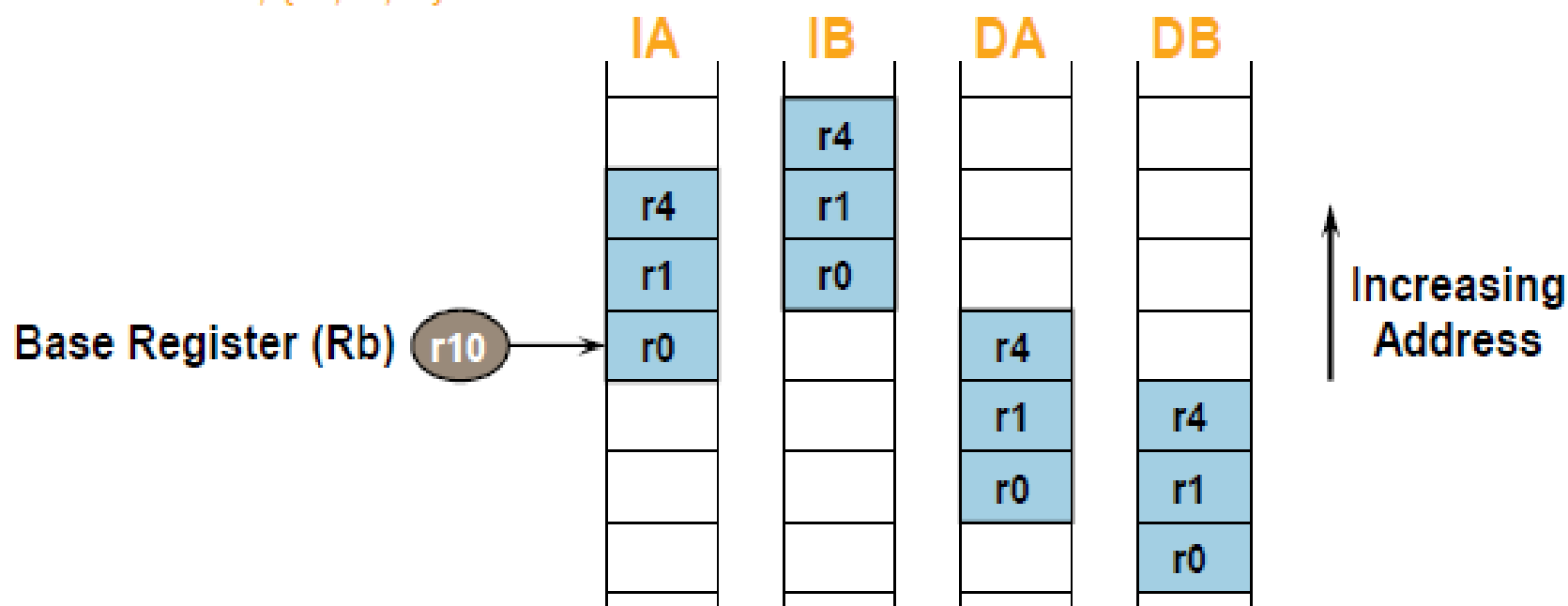
IT       GT              ; IT block with only one conditional instruction
ADDGT   R1, R1, #1        ; Increment R1 conditionally

ITTEE   EQ              ; Next 4 instructions are conditional
MOVEQ   R0, R1           ; Conditional move
ADDEQ   R2, R2, #10       ; Conditional add
ANDNE   R3, R3, #1        ; Conditional AND
BNE.W   dloop            ; Branch instruction can only be used in the last
                          ; instruction of an IT block

IT       NE              ; Next instruction is conditional
ADD      R0, R0, R1       ; Syntax error: no condition code used in IT block.
```

Multiple Register Data Transfer

- These instructions move data between multiple registers and memory
 - `<LDM|STM>{<addressing_mode>}{<cond>} Rb{!}, <register list>`
- 4 addressing modes
 - Increment after (IA) , before (IB)
 - Decrement after (DA) , before (DB)
- Example
 - `LDM r10, {r0,r1,r4}`



Load and Store Multiple Registers

Syntax

op{*addr_mode*}{*cond*} *Rn*{!}, *reglist*

where:

op

Is one of:

LDM

Load Multiple registers.

STM

Store Multiple registers.

addr_mode

This is any one of the following:

IA

Increment address After each access. This is the default.

DB

Decrement address Before each access.

cond

Is an optional condition code

Rn

Specifies the register on which the memory addresses are based.

!

Is an optional writeback suffix. If ! is present the final address, that is loaded from or stored to, is written back into *Rn*.

reglist

Is a list of one or more registers to be loaded or stored, enclosed in braces. It can contain register ranges. It must be comma separated if it contains more than one register or register range

Load and Store Multiple Registers

Restrictions

In these instructions:

- *Rn* must not be PC
- *reglist* must not contain SP
- in any STM instruction, *reglist* must not contain PC
- in any LDM instruction, *reglist* must not contain PC if it contains LR
- *reglist* must not contain *Rn* if you specify the writeback suffix.

When PC is in *reglist* in an LDM instruction:

- bit[0] of the value loaded to the PC must be 1 for correct execution, and a branch occurs to this halfword-aligned address
- if the instruction is conditional, it must be the last instruction in the IT block.

Condition flags

These instructions do not change the flags.

Examples

```
LDM      R8, {R0, R2, R9}      ; LDMIA is a synonym for LDM
STMDB    R1!, {R3-R6, R11, R12}
```

Incorrect examples

```
STM      R5!, {R5, R4, R9} ; Value stored for R5 is unpredictable
LDM      R2, {}             ; There must be at least one register in the list.
```

More Instructions

SXT	Sign - extend 8 / 16 bits to 32 bits
UXT	Zero - extend 8 / 16 bits to 32 bits
SBFX	Signed bit Field Extract
UBFS	Unsigned Bit field Extract
REV	Reverse Byte Order
REV16	Swap Bytes (twice)
REVSH	Reverse and sign-extend
RBIT	Reverse 32 Bits
CLZ	Count Leading Zeros

Example:

Isolated short subroutine call

```
error_queue = xQueueCreate( 10, 1);
```

```
mov    r1, #1  
mov    r2, #0  
mov    r0, #10
```

```
bl      0x115f8          ;<xQueueCreate>
```

```
movw   r3, #4452         ; two 16 bit load ops  
movt   r3, #8192         ; emulate a 32 bit load here !
```

```
str     r0, [r3, #0]
```

Example:

Set bit in I/O location

```
void LedOn(void)
{
    GPIOC->ODR &= ~GPIO_Pin_12;
}
```

LedOn:

mov.w	r3, #4096	; 0x1000
movt	r3, #16385	; 0x4001
ldr	r2, [r3, #12]	
bic.w	r2, r2, #4096	; 0x1000
str	r2, [r3, #12]	
bx	lr	; no stack use – AT ALL!
nop		; word-align next subroutine

Binary -> ASCII conversion

"straight - forward"

```
uint8_t test( uint8_t in)
{
    if(in <= 9)
        return in + '0';
    if( in <= 15)
        return in - 10 + 'a';
    return 'X';
}
```

test:

```
cmp     r0, #9
bls.n   0x88ea <test+18>
cmp     r0, #15
bls.n   0x88e4 <test+12>
```

```
movs    r0, #88    ; 0x58
bx lr
```

```
adds    r0, #87    ; 0x57
uxtb    r0, r0
bx lr
```

```
adds    r0, #48    ; '0'
uxtb    r0, r0
bx lr
```

Binary -> ASCII conversion optimized

```
uint8_t test1( uint8_t in)
{
    if( in <= 15)
        return "01234567890abcdef"[in];
    return 'X';
}
```

```
test:
    cmp     r0, #15
    itte    ls
    ldrls   r3, [pc, #4]
    ldrrls  r0, [r3, r0]
    movhi   r0, #88      ; 'X'

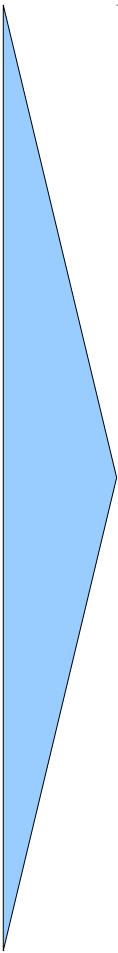
    bx     lr
```

```
.LC0:
    .ascii  "01234567890abcdef\000"
```

CMSIS C-Language Wrappers

Instruction	CMSIS function
CPSIE I	<code>void __enable_irq(void)</code>
CPSID I	<code>void __disable_irq(void)</code>
CPSIE F	<code>void __enable_fault_irq(void)</code>
CPSID F	<code>void __disable_fault_irq(void)</code>
ISB	<code>void __ISB(void)</code>
DSB	<code>void __DSB(void)</code>
DMB	<code>void __DMB(void)</code>
REV	<code>uint32_t __REV(uint32_t int value)</code>
REV16	<code>uint32_t __REV16(uint32_t int value)</code>
REVSH	<code>uint32_t __REVSH(uint32_t int value)</code>
RBIT	<code>uint32_t __RBIT(uint32_t int value)</code>
SEV	<code>void __SEV(void)</code>
WFE	<code>void __WFE(void)</code>
WFI	<code>void __WFI(void)</code>

Lecture Overview

- 
- Introduction, Microcontroller basics
 - 32-bit Microcontroller technology
 - ARM Cortex M Architecture, (CM0 CM3 CM4 CM7)
 - Cortex M Microcontroller Instruction Set
 - Exception Handling, Interrupts
 - Real-time Operating Systems (RTOS)
 - Memory Management, protected memory, MPU
 - Implementing Calculations, Data Formats, FPU
 - I/O, Device drivers
 - Application Design
 - Case Studies, Advanced Design Patterns

<https://skript.eit.h-da.de/ams/>

